Decidability of $\mathcal{ALCI}_{\mathrm{RCC5}}$ via Saturated Split Forests, Patchwork Bags, and Two-Way Parity Automata

A theorem-spine proof with detailed appendices

Consolidated proof manuscript

May 31, 2026

Abstract

This manuscript gives a detailed proof architecture for decidability of concept satisfiability for $\mathcal{ALCI}$ with the five atomic RCC5 relations as roles, interpreted over abstract complete atomic RCC5 frames with strong equality. The proof is organized around three explicit theorems. Theorem A is a split-forest normal-form theorem: every satisfiable concept has a witness-generated saturated split-forest presentation. Theorem B is a finite-abstraction theorem: such presentations are represented exactly, up to the closure of the input concept, by finite patchwork-bag abstractions with exact occurrence identity, relation-vector shadows, and saturated side contexts. Theorem C is an automaton correctness theorem: a finite two-way alternating parity tree automaton accepts exactly the valid finite abstractions. The final decision procedure is then ordinary emptiness for finite two-way alternating parity tree automata.

The paper is intentionally explicit about the role of the original split-forest idea. Split forests remain the semantic normal form and are not replaced by automata. What is replaced is the earlier hand-coded finite certificate layer: regularity, simultaneous eventuality fulfillment, and infinite vertical behavior are handled by automata, while finite RCC5 coherence is handled by complete bag networks and the RCC5 patchwork property. The proof is for abstract composition-table RCC5 frames, not for a fixed topological representation of regions.

Contents

1 Introduction and proof spine **3**
1.1 What survives from the original split-forest idea 4

2 Preliminaries **4**
2.1 Abstract RCC5 frames 4
2.2 Concepts and satisfaction 5
2.3 Closure and types 5
2.4 Two external finite facts 5

3 Theorem A: split-forest normal form **6**
3.1 Witness thinning 6
3.2 Generated containment covers 6
3.3 Side-context saturation 7

4 Theorem B: finite abstraction adequacy **8**
4.1 Patchwork bags 8
4.2 Soundness and completeness of abstractions 9

5 Theorem C: automaton correctness **10**
5.1 Finite alphabet . . . 10
5.2 Two-way alternating parity automaton . . . 10
5.3 Parity condition . . . 10

6 Decidability **11**

A Appendix A: patchwork use and finite-to-countable passage **11**

B Appendix B: witness thinning and generated covers **12**

C Appendix C: saturated side contexts in detail **12**
C.1 Why saturation is necessary . . . 12
C.2 Threshold verticalization . . . 13

D Appendix D: finite abstraction extraction **13**

E Appendix E: patchwork realization of a valid abstraction **14**

F Appendix F: relation-vector shadows **14**

G Appendix G: exact identity and the removal of equality quotients **15**

H Appendix H: automaton details **15**
H.1 Local states . . . 15
H.2 Two-way shadow movement . . . 15
H.3 Request states and parity . . . 16
H.4 Why two-way motion is used . . . 16

I Appendix I: effective bounds **16**

J Appendix J: audit of repaired proof obligations **16**

K Appendix K: explicit decision procedure **17**
K.1 Input normalization . . . 17
K.2 Type enumeration . . . 17
K.3 Ranked context-class enumeration . . . 17
K.4 Alphabet enumeration . . . 18
K.5 Automaton construction . . . 18

L Appendix L: expanded proof of Theorem A **19**
L.1 Step 1: delete irrelevant elements . . . 19
L.2 Step 2: choose generated containment supports . . . 19
L.3 Step 3: split multiple selected superparts . . . 19
L.4 Step 4: insert side witnesses . . . 19
L.5 Step 5: saturate and verticalize . . . 19
L.6 Step 6: prove residual frontier correctness . . . 20
L.7 Step 7: bound the construction . . . 20

M Appendix M: expanded proof of Theorem B **20**
M.1 Extraction direction . . . 20
M.2 Realization direction . . . 20
M.3 Truth lemma: Boolean cases . . . 21
M.4 Truth lemma: existential cases . . . 21
M.5 Truth lemma: universal cases . . . 21

N Appendix N: expanded automaton construction **21**
N.1 State families . . . 21
N.2 Transition clauses . . . 22
N.3 Parity acceptance . . . 22

O Appendix O: diagnostic examples **22**
O.1 Infinite upward chain . . . 22
O.2 Two incomparable superparts . . . 22
O.3 Side witness forced vertical . . . 22
O.4 The diagnostic PPI; PP exclusion . . . 23

P Appendix P: limitations and verification targets **23**

Q Appendix Q: local legality tests in expanded form **23**
Q.1 Network tests . . . 23
Q.2 Type tests . . . 24
Q.3 Witness tests . . . 24
Q.4 Overlap tests . . . 24
Q.5 Connectedness status tests . . . 24
Q.6 Saturation-certificate tests . . . 25

R Appendix R: proof of running-intersection and occurrence identity **25**

S Appendix S: exact use of two-wayness **26**
S.1 Checking an ancestor obligation . . . 26
S.2 Checking sibling interactions . . . 26
S.3 No complexity claim . . . 26

T Appendix T: dependency graph of the proof **27**

U Appendix U: what would invalidate the proof **27**

V Conclusion **27**

1 Introduction and proof spine

The original split-forest intuition is still the key model-theoretic idea. It says that, after irrelevant ambient elements have been removed by witness thinning, the containment part of a model can be represented by a generated vertical forest. Selected PP and PPI witnesses are represented vertically; selected DR and PO witnesses are represented in finite side contexts; multiple incomparable superparts are handled by multiple manifestations of the same occurrence in overlapping finite bags; and strong equality is recovered as exact identity of occurrences, not as a quotient of blocking states.

What changed through the successive repairs is the decision layer. The first hand-written certificate systems tried to make the split forest itself carry all finite coherence, equality, and cyclic eventuality information. That made the proof long and fragile. The present architecture separates the responsibilities:

split forest = semantic normal form, patchwork bags = finite RCC5 abstraction, automaton = regularity

This separation is the main point of the proof.

The proof is organized around the following three theorems.

Theorem A. Every satisfiable concept has a witness-generated saturated split-forest presentation.

Theorem B. Saturated split forests are soundly and completely represented by finite patchwork-bag abstractions.

Theorem C. A finite two-way alternating parity tree automaton accepts exactly the valid patchwork-bag abstractions satisfying the input concept.

The decidability theorem follows immediately from these three statements and the decidability of emptiness for finite two-way alternating parity tree automata.

1.1 What survives from the original split-forest idea

The following table is intended to fix the interpretation of the result.

Original split-forest component	Role in the final proof
Witness-generated models	Survive as the first normalization step.
Generated containment cover tree	Survives. Vertical cover edges are selected generated witness edges, not Hasse edges of an arbitrary starting model.
Split copies / multiple parents	Survive, but are represented as multiple bag manifestations of one occurrence connected by overlap maps.
Sibling and side interfaces	Survive as saturated side contexts inside finite bags.
Residual DR/PO frontier	Survives only after saturation has verticalized all forced PP/PPI/EQ side interactions.
Typed equality quotient	Replaced by exact occurrence identity through connected overlap classes.
Manual blocking and request cycles	Replaced by parity acceptance in a finite two-way automaton.
Pair/triple route transformers	Removed. Finite complete bag networks and the RCC5 patchwork theorem provide global coherence.

This is why the automata proof can be shorter without discarding the split-forest idea. The automaton is not a semantic replacement for the split forest; it is the finite decision engine running over the finite abstraction of split forests.

2 Preliminaries

2.1 Abstract RCC5 frames

Let

$$\mathsf{Rel} = \{\mathsf{EQ}, \mathsf{PP}, \mathsf{PPI}, \mathsf{PO}, \mathsf{DR}\}.$$

The converse operation is given by

$$\mathsf{EQ}^{-1} = \mathsf{EQ}, \quad \mathsf{PP}^{-1} = \mathsf{PPI}, \quad \mathsf{PPI}^{-1} = \mathsf{PP}, \quad \mathsf{PO}^{-1} = \mathsf{PO}, \quad \mathsf{DR}^{-1} = \mathsf{DR}.$$

Definition 2.1 (Abstract complete atomic RCC5 frame)**.** *An abstract complete atomic RCC5 frame is a pair (Δ, ρ) where Δ is a nonempty set and $\rho : \Delta^2 \to \mathsf{Rel}$ satisfies, for all $x, y, z \in \Delta$:*

(R1) $\rho(x, y) = \mathsf{EQ}$ *iff* $x = y$;

(R2) $\rho(y, x) = \rho(x, y)^{-1}$;

(R3) $\rho(x, z) \in \mathrm{Comp}(\rho(x, y), \rho(y, z))$.

We use the standard RCC5 composition table:

Comp	EQ	PP	PPI	PO	DR
EQ	{EQ}	{PP}	{PPI}	{PO}	{DR}
PP	{PP}	{PP}	Rel	{PP, PO, DR}	{DR}
PPI	{PPI}	{EQ, PP, PPI, PO}	{PPI}	{PPI, PO}	{PPI, PO, DR}
PO	{PO}	{PP, PO}	{PPI, PO, DR}	Rel	{PPI, PO, DR}
DR	{DR}	{PP, PO, DR}	{DR}	{PP, PO, DR}	Rel

Remark 2.2 (Scope)**.** *The theorem proved here is for abstract frames satisfying the RCC5 composition table and the patchwork property stated below. It is not a representation theorem for regular closed regions in a fixed topological space.*

2.2 Concepts and satisfaction

Concepts are formed from concept names, Boolean connectives, and modalities $\exists R.C$ and $\forall R.C$ for $R \in \mathsf{Rel}$. An interpretation $\mathcal{I}$ is an abstract RCC5 frame plus an assignment of concept names to subsets of its domain. Satisfaction is standard:

$$\mathcal{I}, x \models \exists R.C \quad \iff \quad \text{there is } y \text{ with } \rho(x, y) = R \text{ and } \mathcal{I}, y \models C.$$

Universal restrictions are interpreted dually. Since EQ is identity,

$$\exists \mathsf{EQ}.C \equiv C, \qquad \forall \mathsf{EQ}.C \equiv C.$$

We normalize such formulas away.

2.3 Closure and types

Put concepts in negation normal form. Let $\overline{C}$ denote the NNF complement of C.

Definition 2.3 (Closure)**.** *For an input concept C_0, $\mathrm{Cl}(C_0)$ is the least finite set containing C_0, closed under subformulas, under $C \mapsto \overline{C}$, and under replacing $\exists \mathsf{EQ}.C$ and $\forall \mathsf{EQ}.C$ by C.*

Definition 2.4 (Type)**.** *A type over* $\mathrm{Cl}(C_0)$ *is a set* $t \subseteq \mathrm{Cl}(C_0)$ *such that:*

(T1) for each $C \in \mathrm{Cl}(C_0)$ *exactly one of* $C, \overline{C}$ *lies in* t*;*

(T2) $C \sqcap D \in t$ *iff* $C \in t$ *and* $D \in t$*;*

(T3) $C \sqcup D \in t$ *iff* $C \in t$ *or* $D \in t$*;*

(T4) normalized EQ*-clauses are respected.*

Let $\mathsf{Typ}(C_0)$ *be the finite set of all such types.*

For an element x in an interpretation, $\mathrm{tp}(x)$ denotes the type of closure formulas true at x.

2.4 Two external finite facts

The proof uses two standard finite facts. They are stated explicitly because all other constructions are internal to the proof.

Theorem 2.5 (RCC5 patchwork property)**.** *Let* N_1, N_2 *be finite complete atomic RCC5 networks over variable sets* V_1, V_2*. Assume that each* N_i *is satisfiable as an abstract RCC5 network and that* N_1, N_2 *agree on* $V_1 \cap V_2$*. Then their union has a satisfiable completion on* $V_1 \cup V_2$*. The same holds for any finite tree-shaped family of finite complete atomic RCC5 networks agreeing on overlaps. A countable tree-shaped family has a countable abstract completion all of whose finite restrictions are obtained by finite patchwork.*

Remark 2.6. *For the abstract composition-table semantics used here, Theorem 2.5 is the algebraic input that replaces all route-dependent pair/triple transformer claims. Appendix A spells out exactly how it is used and why a countable tree of bags follows from the finite patchwork theorem.*

Theorem 2.7 (Two-way alternating parity tree automata)**.** *Emptiness for finite two-way alternating parity tree automata over finite ranked alphabets is decidable.*

3 Theorem A: split-forest normal form

3.1 Witness thinning

Lemma 3.1 (Witness-generated submodel)**.** *If* $\mathcal{I}, a_0 \models C_0$*, then there is a subinterpretation* $\mathcal{I} \restriction W$ *with* $a_0 \in W$ *such that:*

(i) W *is generated from* a_0 *by choosing one witness for every true existential formula* $\exists R.C \in \mathrm{Cl}(C_0)$ *at each generated element;*

(ii) for every $a \in W$ *and* $C \in \mathrm{Cl}(C_0)$*,*

$$\mathcal{I}, a \models C \quad \Longleftrightarrow \quad \mathcal{I} \restriction W, a \models C.$$

Proof. Let $W_0 = \{a_0\}$. Having constructed W_i, for each $a \in W_i$ and each existential formula $\exists R.C \in \mathrm{Cl}(C_0)$ true at a in $\mathcal{I}$, choose one witness b with $\rho(a, b) = R$ and $\mathcal{I}, b \models C$, and put it into W_{i+1}. Let $W = \bigcup_i W_i$.

The equivalence of truth is by induction on closure formulas. Atomic and Boolean cases are immediate. If $C = \exists R.D$, then any restricted-model witness is a full-model witness. Conversely,

if $\mathcal{I}, a \models \exists R.D$, the chosen witness for that existential belongs to W and satisfies D in $\mathcal{I}$; by the induction hypothesis it satisfies D in the restricted model. For $C = \forall R.D$, the forward direction is preserved by deleting elements. Conversely, if $\mathcal{I}, a \not\models \forall R.D$, then $\mathcal{I}, a \models \exists R.\overline{D}$. Since the closure is closed under NNF complements, a selected witness for $\exists R.\overline{D}$ lies in W, and by induction it still satisfies $\overline{D}$ in the restriction. Hence the universal fails in the restriction. This proves the claim. □

3.2 Generated containment covers

A generated witness edge is not a Hasse edge of the original model. It is a selected edge retained by the thinning construction. The semantic PP and PPI relations along the presentation are obtained by vertical closure plus the finite bag networks.

Definition 3.2 (Generated containment skeleton)**.** *A generated containment skeleton is a forest-like structure with vertical cover edges directed downward as selected* PPI *covers, equivalently upward as selected* PP *covers. Vertical rays may be infinite. Repetition of a finite profile along a ray is blocking, not semantic equality.*

Definition 3.3 (Saturated split-forest presentation)**.** *A saturated split-forest presentation consists of:*

(SF1) a tree decomposition of occurrences into finite bags;

(SF2) vertical containment edges inside and between bags representing selected PP/PPI *witnesses;*

(SF3) overlap maps identifying multiple manifestations of the same occurrence;

(SF4) finite side contexts for selected DR/PO *witnesses;*

(SF5) a saturation operation that verticalizes every side/tree or side/side pair labelled PP*,* PPI*, or* EQ *before the pair may be forgotten;*

(SF6) residual frontier pairs labelled only DR *or* PO *after saturation;*

(SF7) selected supports for vertical eventualities.

3.3 Side-context saturation

The role of side-context saturation is to avoid the earlier error in which a DR or PO witness was placed in a side bag although it was forced by composition to be vertically comparable with a boundary node.

Definition 3.4 (Ranked side context)**.** *For a source occurrence u and a formula occurrence that triggers a side witness, the rank of the side context is the modal depth remaining below that triggering formula. Rank decreases when saturation recursively inserts side contexts for witnesses generated inside the current side context.*

Definition 3.5 (Saturation rules)**.** *A side context for u is saturated if it is closed under the following finite rules until no rule applies at the current rank.*

(S1) Insert every selected DR *or* PO *witness required at u.*

(S2) Insert the relevant vertical boundary of u: its live parent, live children, selected super-part/subpart support nodes, and overlap manifestations.

(S3) For every side/tree or side/side pair whose inherited or locally declared relation is PP*,* PPI*, or* EQ*, insert the corresponding local vertical stratum or overlap identity.*

(S4) For an infinite ancestor or descendant tail forced by composition, do not try to store the whole tail in a bounded side bag. Instead splice the witness into the generated containment backbone at a bounded threshold and record the remaining tail as a vertical support request.

(S5) Recursively saturate side witnesses created inside the context at strictly smaller rank.

(S6) Only after all previous rules have been exhausted may a forgotten frontier pair be treated as residual; residual frontier labels are restricted to DR *and* PO*.*

Lemma 3.6 (Effective finite bound for saturated contexts)**.** *For every input concept C_0 there is a computable number $K(C_0)$ such that every rank-bounded saturated side context needed for C_0 has a representative using at most $K(C_0)$ ports, up to equality of finite labelled context type.*

Proof. Let $n = |\operatorname{Cl}(C_0)|$ and $T = |\mathsf{Typ}(C_0)|$. For each integer k, there are finitely many labelled finite contexts on at most k ports: at most 5^{k^2} RCC5 networks, T^k type assignments, and finitely many witness, overlap, shadow, and request declarations. The saturation rules are decidable on such finite labelled objects.

Define the set of rank-0 context classes by enumeration: rank 0 has no recursive side subcontexts, so one enumerates finite labelled contexts, keeps those satisfying local RCC5 consistency and saturation, and quotients by isomorphism preserving ports, types, networks, and declarations. This is finite.

Assume rank-r classes have been effectively enumerated. A rank-$(r+1)$ context is described by a finite local bag together with, for each recursive side trigger, one of the finitely many rank-r classes. Hence the collection of rank-$(r+1)$ classes is finite and effectively enumerable. Since ranks are bounded by n, the union of all rank classes up to n is finite. Let $K(C_0)$ be the maximum number of ports in chosen representatives of these classes. This gives an effective bound. □

Theorem 3.7 (A: split-forest normal form)**.** *If C_0 is satisfiable over abstract complete atomic RCC5 frames, then C_0 is satisfiable in a witness-generated saturated split-forest presentation of width at most $K(C_0)$.*

Proof. Start from a satisfying model and apply Lemma 3.1. Retain the selected PP/PPI witnesses as generated vertical supports. If a generated occurrence requires several incomparable superpart witnesses, represent the occurrence in several adjacent bags whose overlap maps identify the manifestations as one occurrence. This avoids assigning several parents to a single tree node and avoids nontrivial semantic EQ edges.

For every selected DR/PO witness, create a side context and saturate it by rules (S1)–(S6). If composition with a vertical boundary forces the side witness into a vertical relation with some boundary node, insert a local vertical stratum. If this relation persists into an infinite ancestor or descendant tail, splice the side witness into the vertical backbone at a bounded threshold and record the remainder as a vertical support request. The recursive part of this construction is rank-decreasing and therefore terminates within the bound of Lemma 3.6.

Every finite bag is labelled by the restriction of the original model's RCC5 network and by the truth types of its occurrences. Adjacent bags agree on overlaps because they represent the same original occurrences. Thus the resulting presentation is saturated, witness-generated, and has width at most $K(C_0)$. □

4 Theorem B: finite abstraction adequacy

4.1 Patchwork bags

Definition 4.1 (Patchwork bag)**.** *A patchwork bag over C_0 and width K is a tuple*

$$B = (P, N, \tau, \mu, W, U, \Gamma)$$

where:

(B1) $P \subseteq \{1, \dots, K\}$ *is a finite port set;*

(B2) $N : P^2 \to \mathsf{Rel}$ *is a complete atomic RCC5 network with* $N(p,p) = \mathsf{EQ}$, $N(p,q) \neq \mathsf{EQ}$ *for* $p \neq q$, *converse symmetry, and the composition table on every triple;*

(B3) $\tau : P \to \mathsf{Typ}(C_0)$ *assigns types;*

(B4) μ *is a finite family of partial injective overlap maps to the parent and child positions;*

(B5) W *records local witnesses for existential formulas;*

(B6) U *records relation-vector shadows for universal obligations;*

(B7) Γ *records vertical request declarations for nonlocal* PP/PPI *eventualities.*

Definition 4.2 (Overlap identity)**.** *Given a tree of bags, global occurrences are equivalence classes of local port occurrences (t, p) under the equivalence relation generated by the overlap maps. Equality is exact occurrence identity:*

$$x \mathsf{EQ} y \quad \textit{iff} \quad x = y \textit{ as overlap-equivalence classes.}$$

No two distinct local ports in one bag may be labelled EQ.

Definition 4.3 (Relation-vector shadow)**.** *A shadow profile has the form*

$$(t, \vec{r}, \Omega),$$

where $t \in \mathsf{Typ}(C_0)$ is the type of a source occurrence, $\vec{r} \in \mathsf{Rel}^P$ is the vector of relations from the source to the current live ports, and $\Omega \subseteq \mathsf{Rel} \times \mathrm{Cl}(C_0)$ is the set of universal obligations induced by formulas $\forall R.D$ in the source type. The obligation (R, D) says: every target in relation R to the source must have D in its type.

Definition 4.4 (Valid patchwork abstraction)**.** *A valid patchwork abstraction for C_0 is a finitely branching tree labelled by patchwork bags of width at most $K(C_0)$ such that:*

(V1) every bag is locally RCC5-consistent and type-consistent;

(V2) adjacent bags agree exactly on overlap ports, types, internal relations, and relevant witness/shadow declarations;

(V3) overlap classes are connected subtrees;

(V4) every existential formula has a local witness or a declared vertical request;

(V5) every universal formula is represented by a shadow and every introduced target is checked against exact relation vectors;

(V6) side contexts are saturated according to rules (S1)–(S6);

(V7) residual frontier pairs are only DR *or* PO*;*

(V8) vertical requests are tied to a concrete support branch and to the same overlap-equivalence class of the source occurrence.

4.2 Soundness and completeness of abstractions

Theorem 4.5 (B: finite abstraction adequacy)**.** *For every C_0:*

(a) every saturated split-forest presentation of width $K(C_0)$ induces a valid patchwork abstraction;

(b) every valid patchwork abstraction unfolds to an abstract complete atomic RCC5 interpretation satisfying exactly the types and witness/universal obligations represented in the abstraction.

Proof. For (a), label every bag of the split-forest presentation by the finite network inherited from the underlying model, the truth type of each occurrence, the overlap maps, the selected local witness declarations, the vertical request declarations, and the universal shadows. Because the split forest is saturated, every side/tree or side/side pair labelled PP, PPI, or EQ has already been represented by a local vertical stratum or overlap identity; hence residual forgotten frontier pairs are DR or PO. All validity clauses are immediate from the construction.

For (b), take the set V of global occurrences to be the quotient of local ports by overlap maps. Each bag supplies a finite complete RCC5 network on the occurrences it contains. Adjacent bags agree on overlaps, and occurrence classes are connected. By Theorem 2.5, the tree of bag networks has a global abstract RCC5 completion ρ on V.

Interpret concept names by the bag types. This is well-defined because types agree on overlaps. Boolean closure follows from type consistency. For existential formulas, use the recorded local witness if present; otherwise the formula is a vertical PP/PPI eventuality and is fulfilled by the declared support branch. For universal formulas, let x be a source occurrence with $\forall R.D$ in its type and let y be any target occurrence with $\rho(x, y) = R$. Choose bags containing x and y and the finite connecting subtree between them. The shadow for x is propagated along this connecting subtree. When y is introduced, the exact relation vector from x to the live ports includes $\rho(x, y) = R$; validity therefore forces D into the type of y. Thus universal restrictions are satisfied. Strong equality holds because EQ is only identity of overlap classes. Hence the unfolded interpretation realizes the abstraction. □

5 Theorem C: automaton correctness

5.1 Finite alphabet

Let $K = K(C_0)$. The alphabet Σ_{C_0} consists of all patchwork bags of width at most K satisfying the finite local tests in Definition 4.1. It is effectively enumerable because $P \subseteq \{1, \dots, K\}$, N ranges over 5^{K^2} relation labellings, τ over $|\mathsf{Typ}(C_0)|^K$ type assignments, and all witness, shadow, overlap, and request declarations range over finite powersets.

5.2 Two-way alternating parity automaton

Definition 5.1 (Automaton $\mathcal{A}_{C_0}$)**.** *The automaton $\mathcal{A}_{C_0}$ runs on ranked trees over Σ_{C_0}. Its finite state set has the following components:*

(Q1) local states checking bag consistency, type consistency, and side saturation;

(Q2) overlap states checking parent/child overlap agreement and connectedness of occurrence classes;

(Q3) witness states checking local existential witnesses;

(Q4) shadow states carrying relation-vector shadows two-way through the tree;

(Q5) request states following vertical support branches for PP/PPI *eventualities;*

(Q6) patchwork states checking finite complete bag networks and overlap agreement.

The transition relation is finite because all referenced ports, types, relations, formulas, shadows, and request declarations belong to finite sets.

5.3 Parity condition

For each request kind (R, D) with $R \in \{\mathsf{PP}, \mathsf{PPI}\}$ and $D \in \mathrm{Cl}(C_0)$, a request state is created whenever a port type contains $\exists R.D$ without a local witness. The state follows the declared support branch while carrying the source occurrence through overlap maps or shadows. It is fulfilled when it reaches a port q whose exact recorded relation to the source is R and whose type contains D. The finitely many request kinds give a generalized Buchi condition, converted effectively to a parity condition.

Lemma 5.2 (Automaton soundness)**.** *If $\mathcal{A}_{C_0}$ accepts a labelled tree, then the tree is a valid patchwork abstraction with a root port whose type contains C_0.*

Proof. The local states enforce local RCC5 and type consistency. Overlap states enforce agreement and connected occurrence classes. Witness states enforce local existential witnesses. Shadow states enforce exact universal propagation: whenever a target is introduced and the carried relation vector has relation R to the source, every obligation (R, D) forces D at the target. Request states and parity acceptance enforce fulfillment of all vertical eventualities. Saturation checks enforce verticalization of all forced nonresidual side interactions. These are precisely the validity clauses in Definition 4.4. □

Lemma 5.3 (Automaton completeness)**.** *Every valid patchwork abstraction for C_0 is accepted by $\mathcal{A}_{C_0}$.*

Proof. Use the accepting run that follows the witnesses, overlaps, shadows, and vertical support branches recorded in the abstraction. Every local and overlap check succeeds by validity. Shadow runs succeed because valid abstractions carry exact relation vectors. Each vertical request has a declared support branch and endpoint witness, so the parity condition is satisfied. Therefore the automaton accepts. □

Theorem 5.4 (C: automaton correctness)**.** *For every C_0,*

$$L(\mathcal{A}_{C_0}) \neq \varnothing \quad \Longleftrightarrow \quad C_0 \text{ has a valid patchwork abstraction.}$$

Proof. Immediate from Lemmas 5.2 and 5.3. □

6 Decidability

Theorem 6.1 (Decidability)**.** *Concept satisfiability for $\mathcal{ALCI}_{\mathrm{RCC5}}$ over abstract complete atomic RCC5 frames with strong equality is decidable.*

Proof. Given C_0, compute $\mathrm{Cl}(C_0)$, $\mathsf{Typ}(C_0)$, the finite context bound $K(C_0)$, the finite alphabet Σ_{C_0}, and the two-way alternating parity tree automaton $\mathcal{A}_{C_0}$. By Theorem 3.7, satisfiability implies existence of a saturated split-forest presentation. By Theorem 4.5, this is equivalent to existence of a valid patchwork abstraction. By Theorem 5.4, this is equivalent to nonemptiness of $\mathcal{A}_{C_0}$. Emptiness for finite two-way alternating parity tree automata is decidable. Hence satisfiability is decidable. □

A Appendix A: patchwork use and finite-to-countable passage

This appendix records the exact form in which RCC5 patchwork is used.

Definition A.1 (Finite complete atomic network)**.** *A finite complete atomic RCC5 network on a finite variable set V is a map $N : V^2 \to \mathsf{Rel}$ satisfying $N(x, x) = \mathsf{EQ}$, $N(x, y) = N(y, x)^{-1}$, and $N(x, y) \neq \mathsf{EQ}$ when $x \neq y$. It is composition-closed if $N(x, z) \in \mathrm{Comp}(N(x, y), N(y, z))$ for all $x, y, z \in V$.*

A bag network in this proof is always finite, complete, and composition-closed. A family of bag networks is tree-shaped if the bags are indexed by nodes of a tree and every occurrence appears in a connected set of bags.

Lemma A.2 (Finite tree patching from two-bag patchwork)**.** *Assume the two-bag patchwork property of Theorem 2.5. Then every finite tree-shaped family of complete atomic RCC5 bag networks agreeing on overlaps has a satisfiable completion.*

Proof. Induct on the number of bags. If there is one bag, there is nothing to prove. Otherwise choose a leaf bag L with neighbor M. By induction, the remaining tree has a satisfiable completion N_R. The leaf network N_L agrees with N_R on the overlap $L \cap M$, because it agrees with M and M is included in the remainder. Apply the two-bag patchwork property to N_L and N_R to obtain a completion of their union. This completes the induction. □

Lemma A.3 (Countable patching)**.** *A countable tree-shaped family of finite complete bag networks agreeing on overlaps has a countable abstract completion if every finite subtree of bags has one.*

Proof. Let V be the countable set of global occurrences. The product space Rel^{V^2} is compact because Rel is finite and discrete. For each finite subtree S of bags, let C_S be the closed set of all total pair labellings extending the patchwork completion on the occurrences occurring in S and satisfying the composition constraints on those occurrences. The family $\{C_S\}$ has the finite intersection property: finitely many finite subtrees are contained in a larger finite subtree, which has a patchwork completion. Hence the intersection is nonempty. Any element of the intersection is a total RCC5 labelling extending every bag. Since every triple of occurrences occurs in some finite subset, all composition constraints hold. □

B Appendix B: witness thinning and generated covers

The thinning lemma is what makes dense or extraneous ambient structure irrelevant. If the original model contains intermediate points between a source and a selected witness, those points are not

retained unless they are themselves needed as selected witnesses. The generated cover relation is therefore chosen by the construction; it is not the Hasse diagram of the original model.

Lemma B.1 (Generated cover preservation)**.** *Let W be the witness-generated subdomain from Lemma 3.1. For every existential formula $\exists R.C \in \mathrm{Cl}(C_0)$ true at $x \in W$, there is a selected generated edge from x to a witness $y \in W$ with $\rho(x, y) = R$ and $C \in \mathrm{tp}(y)$.*

Proof. This is exactly the construction of W. The selected edge is a bookkeeping edge, not an additional semantic relation. Its label is justified by the semantic relation $\rho(x, y)$ in the restricted model. □

Lemma B.2 (Vertical closure for PP/PPI)**.** *In a generated containment presentation, if $x = x_0, x_1, \ldots, x_m = y$ is a vertical chain of selected* PP*-cover edges upward, then $\rho(x, y) =$* PP*. Dually, a downward chain of selected* PPI*-cover edges gives* PPI*.*

Proof. Each upward step is labelled PP. Since $\mathrm{Comp}(\mathsf{PP}, \mathsf{PP}) = \{\mathsf{PP}\}$, induction on the chain length gives $\rho(x_0, x_m) = \mathsf{PP}$. The dual statement follows from converse symmetry and $\mathrm{Comp}(\mathsf{PPI}, \mathsf{PPI}) = \{\mathsf{PPI}\}$. □

C Appendix C: saturated side contexts in detail

This appendix expands the construction behind Lemma 3.6.

C.1 Why saturation is necessary

Suppose $u\mathsf{PO}w$ and $u\mathsf{PP}b$. Then the relation between w and b is constrained by

$$\rho(w, b) \in \mathrm{Comp}(\mathsf{PO}, \mathsf{PP}) = \{\mathsf{PP}, \mathsf{PO}\}.$$

Thus w may be a proper part of b. Similarly, if $u\mathsf{PO}w$ and $b\mathsf{PP}u$, then

$$\rho(b, w) \in \mathrm{Comp}(\mathsf{PP}, \mathsf{PO}) = \{\mathsf{PP}, \mathsf{PO}, \mathsf{DR}\}.$$

The side witness w cannot be safely treated as an unstructured residual frontier point until all such forced vertical possibilities have been checked. Saturation is precisely the closure operation that performs this check.

C.2 Threshold verticalization

When a side witness is forced into a vertical relation with an infinite ancestor or descendant tower, bounded bags cannot contain the whole tower. The repair is threshold verticalization. If a side witness w becomes eventually comparable with a vertical ray, choose the first relevant boundary point up to type/context equivalence and attach a manifestation of w at that threshold. The remaining infinite tail is represented as a vertical request tracked by the automaton. Since there are only finitely many boundary/context types, such a threshold can be chosen from a bounded finite set of representatives.

Lemma C.1 (Saturation soundness)**.** *If a side context is saturated, then every forgotten pair involving a side occurrence and a boundary occurrence is either already represented by a vertical stratum/overlap identity or has label* DR *or* PO*.*

Proof. By saturation rule (S3), any pair labelled PP, PPI, or EQ is not forgotten as residual; it is represented explicitly. Therefore a pair that remains eligible to be forgotten cannot have one of those labels. Since the RCC5 atoms are exactly $\mathsf{EQ}, \mathsf{PP}, \mathsf{PPI}, \mathsf{PO}, \mathsf{DR}$, its label is PO or DR. □

Lemma C.2 (Saturation completeness)**.** *Let a side witness occur in a witness-generated model. The saturated context construction can represent all relations between that witness and the bounded vertical boundary relevant to formulas in* $\mathrm{Cl}(C_0)$*.*

Proof. Insert the witness and boundary points. For each pair, inspect the inherited RCC5 label from the model. If it is DR or PO, it may remain as a side relation. If it is EQ, connect the manifestations by overlap identity. If it is PP or PPI, insert the finite vertical stratum witnessing that containment at the current rank. If the relation interacts with an infinite vertical ray, use threshold verticalization and transfer the tail to a vertical request. Recursive side obligations are generated only from proper subformulas, so rank decreases. Therefore the process terminates and reproduces every relevant inherited relation. □

D Appendix D: finite abstraction extraction

Given a saturated split forest, the extraction of a valid patchwork abstraction is mechanical.

(E1) For every finite bag in the split forest, take its port set as P.

(E2) Let $N(p, q)$ be the inherited RCC5 relation between the corresponding occurrences.

(E3) Let $\tau(p)$ be the closure type of the occurrence.

(E4) Record overlap maps exactly as the tree decomposition identifies occurrences across adjacent bags.

(E5) For each existential formula, record the selected same-bag witness or the vertical support request.

(E6) For every live source with universals, record its relation vector to the current bag ports and the corresponding obligation set.

(E7) Retain only saturated side contexts; by Appendix C the residual frontier property holds.

Each step is finite and invariant under isomorphism of finite labelled bags. Thus the extraction lands in the finite alphabet Σ_{C_0}.

E Appendix E: patchwork realization of a valid abstraction

Let T be a valid patchwork abstraction. Define

$$V_T = \{(t, p) : t \in T, p \in P_t\}/ \equiv_{\mathrm{ov}},$$

where $\equiv_{\mathrm{ov}}$ is generated by overlap maps. For every node t, the bag network N_t induces a complete network on the corresponding subset of V_T.

Lemma E.1 (Bag agreement)**.** *If two adjacent bags contain the same two global occurrences, then they assign the same RCC5 relation to them.*

Proof. The two local manifestations lie in the overlap domain. Validity requires overlap maps to preserve the full finite network on the overlap, hence the relation agrees. □

Lemma E.2 (Global realization)**.** *There is an abstract complete atomic RCC5 relation ρ_T on V_T extending all bag networks.*

Proof. Apply the countable patching lemma from Appendix A to the tree-shaped family of finite bag networks. The connectedness of overlap classes gives the running-intersection property needed for tree-shaped patching. The resulting total labelling extends every bag network and satisfies all RCC5 composition constraints on every triple. □

Lemma E.3 (Truth lemma)**.** *For every global occurrence x represented in the abstraction and every $C \in \mathrm{Cl}(C_0)$,*

$$C \in \mathrm{tp}(x) \quad \Longleftrightarrow \quad \mathcal{I}_T, x \models C,$$

where $\mathcal{I}_T$ is the interpretation induced by ρ_T and the bag types.

Proof. The proof is by induction on C. Atomic and Boolean cases follow from the type definition. Existentials are satisfied by recorded local witnesses or by fulfilled vertical requests. For universals, let $\forall R.D \in \mathrm{tp}(x)$ and suppose $\rho_T(x, y) = R$. Choose a finite connected subtree of bags containing a manifestation of x, a manifestation of y, and all bags on the path between them. The shadow for x propagates along this path with exact relation vectors. When y is introduced or live, the relation vector entry to y is R, so validity forces $D \in \mathrm{tp}(y)$. By induction, $\mathcal{I}_T, y \models D$.

Conversely, if $\forall R.D \notin \mathrm{tp}(x)$, then by closure and type completeness $\exists R.\overline{D} \in \mathrm{tp}(x)$, and the existential clause supplies a witness y with relation R and $\overline{D} \in \mathrm{tp}(y)$. By induction D fails at y, so the universal is false. The existential converse is similar. □

F Appendix F: relation-vector shadows

Relation-vector shadows are the part of the finite alphabet that prevents universal obligations from being lost when the source occurrence leaves the current bag.

Lemma F.1 (Shadow sufficiency)**.** *Let x be a source occurrence whose type is t and whose obligations are Ω. Suppose two partial decompositions have the same shadow $(t, \vec{r}, \Omega)$ at a boundary bag. Then every extension bag imposes the same universal type requirements on newly introduced ports in both decompositions.*

Proof. A newly introduced port p is checked only through its recorded relation from the source and the obligations in Ω. Both decompositions have the same obligations and the same relation vector on the boundary. Valid transitions require the new relation vector to agree with the boundary vector on overlap and to be locally RCC5-patchable with the new bag. Thus the set of requirements of the form $D \in \tau(p)$ is determined entirely by $(t, \vec{r}, \Omega)$ and the new bag label. □

Remark F.2. *This is an exact-vector mechanism, not a may-analysis. The automaton guesses exact finite vectors that must be patchwork-realizable. Soundness uses patchwork realization; completeness chooses the vectors from a real split forest.*

G Appendix G: exact identity and the removal of equality quotients

Earlier versions used typed equality ports and then quotienting. The present proof avoids that layer.

Lemma G.1 (Strong equality)**.** *In the realization of a valid patchwork abstraction, $x\mathsf{EQ}y$ iff $x = y$ as overlap-equivalence classes.*

Proof. Inside a bag, the finite network requires $N(p, q) = \mathsf{EQ}$ iff $p = q$. Across bags, two manifestations represent the same occurrence only if they are related by the overlap equivalence relation. The global domain is the quotient by this relation. Thus equality in the domain is exactly overlap identity. No patchwork completion may label two distinct global occurrences by EQ, because every finite network and every overlap network enforces strong equality, and the compactness construction includes the constraints $\rho(x, y) \neq \mathsf{EQ}$ for distinct occurrence classes. □

Remark G.2. *Blocking repetition is now harmless. Two occurrences in different repetitions of the same finite profile are distinct global occurrence classes unless connected by overlap maps. Therefore a one-state vertical loop represents infinitely many fresh occurrences, not an EQ-cycle.*

H Appendix H: automaton details

This appendix expands Theorem C.

H.1 Local states

A local state checks the finite tests:

(L1) $N(p, p) = \mathsf{EQ}$ and $N(p, q) \neq \mathsf{EQ}$ for $p \neq q$;

(L2) converse symmetry;

(L3) triple composition closure;

(L4) Boolean type consistency;

(L5) local witness correctness;

(L6) side-context saturation flags.

All are finite table lookups.

H.2 Two-way shadow movement

A shadow state stores $(p, t, \vec{r}, \Omega)$, where p is a current local manifestation if the source is live, or a marker that the source is represented only by the shadow. The transition may move to the parent or a child. Across an overlap, it updates live port numbers by the overlap injection. If the source is not live, it keeps t and Ω and updates $\vec{r}$ using the relation vector declared in the adjacent bag label. The transition is allowed only if the old and new vectors agree on shared ports and are compatible with the bag network.

H.3 Request states and parity

For each $e = (R, D)$ with $R \in \{\mathsf{PP}, \mathsf{PPI}\}$, define request states q_e. A transition from q_e either reaches a witness port and enters a fulfilled state, or follows the declared support branch. The generalized Buchi condition is:

for every $e, exteveryinfinitepathwithinfinitelymanypendingq_e$ has infinitely many fulfillments of e.

This is converted to a parity condition by the standard round-robin counter over the finite set of request kinds. The number of priorities is at most $2m + 1$, where

$$m \leq 2 \cdot |\operatorname{Cl}(C_0)|.$$

H.4 Why two-way motion is used

One-way ancestor summaries require a proof that all necessary ancestor and sibling information has been losslessly summarized. The present proof avoids this burden: the automaton is permitted to move to the parent, child, and back along already traversed interfaces. This does not change decidability, since two-way alternating parity tree automata have decidable emptiness, but it makes the proof of universal propagation and overlap identity direct.

I Appendix I: effective bounds

Let $n = |\operatorname{Cl}(C_0)|$ and $T = |\mathsf{Typ}(C_0)| \leq 2^n$. Let $K = K(C_0)$ be the saturated context width from Lemma 3.6. Then:

- the number of port sets is at most 2^K;
- the number of atomic networks on K ports is at most 5^{K^2};
- the number of type assignments is at most T^K;
- the number of relation-vector shadows is at most $T \cdot 5^K \cdot 2^{5n}$;
- witness, request, and overlap declarations are finite powersets over sets bounded by a computable function of K, n, T.

Thus the alphabet Σ_{C_0} is effectively enumerable. The automaton state set is bounded by a computable expression polynomial in the alphabet size times the number of closure formulas and shadows. No optimized complexity claim is made; decidability only requires effective finiteness.

J Appendix J: audit of repaired proof obligations

Obligation	Discharge
Witness thinning	Lemma 3.1 and Appendix B.
Generated split forest	Theorem 3.7; generated edges are selected witnesses, not original Hasse covers.
Multiple superparts	Represented by several bag manifestations of one occurrence via overlap maps.

Side/tree defects	Saturation rules (S1)–(S6), Appendix C, and threshold verticalization.
Residual DR/PO frontier	Derived only after saturation; never assumed at insertion time.
Pair/triple coherence	Removed from route transformers; supplied by complete bag networks plus Theorem 2.5.
Strong equality	Exact overlap identity; Appendix G.
Universal propagation	Exact relation-vector shadows; Appendix F and Appendix H.
Vertical eventualities	Two-way parity request states; Appendix H.
Effective finite search	Alphabet and state bounds; Appendix I.

K Appendix K: explicit decision procedure

This appendix writes the decision procedure as an ordinary finite algorithm. The theorem proof can be read as the correctness proof of this algorithm.

K.1 Input normalization

Given a concept C_0, first put it in negation normal form. Replace every occurrence of $\exists \mathsf{EQ}.C$ and $\forall \mathsf{EQ}.C$ by C. This is correct because in the intended semantics EQ is identity. Then compute $\mathrm{Cl}(C_0)$. The closure contains all formulas on which the automaton ever branches. No formula outside the closure is inspected.

K.2 Type enumeration

Enumerate all subsets $t \subseteq \mathrm{Cl}(C_0)$ and keep exactly those satisfying the Boolean consistency tests from the definition of type. This yields the finite set $\mathsf{Typ}(C_0)$. A type is not required to be globally realizable at this stage. Global realizability is checked later by bag consistency, shadow propagation, patchwork, and parity acceptance.

K.3 Ranked context-class enumeration

The following procedure computes the finite set of side-context classes used to obtain $K(C_0)$.

(K1) For rank 0, enumerate all finite port sets P up to increasing size m, all complete atomic networks on P, all type assignments $P \to \mathsf{Typ}(C_0)$, all witness declarations whose target formula has no nested modal obligations, and all overlap declarations. Keep the objects that pass local RCC5 consistency and saturation with no recursive side subcontext.

(K2) Quotient the retained objects by isomorphism of labelled port structures. Since the labels range over finite sets and rank 0 has no recursive parameter, the quotient is finite.

(K3) Having enumerated rank r classes, enumerate rank $r+1$ objects by allowing each recursive side trigger to name one of the finitely many rank r classes. Keep only saturated objects.

(K4) Continue up to rank $|\,\mathrm{Cl}(C_0)|$.

The proof of Lemma 3.6 is exactly the termination proof for this enumeration. In implementation one may enumerate by increasing width and stop when the closure operator on rank classes reaches a fixed point. The paper does not claim an optimal bound; it only uses the existence of an effective bound.

K.4 Alphabet enumeration

With $K = K(C_0)$ fixed, enumerate all candidate bag labels

$$(P, N, \tau, \mu, W, U, \Gamma)$$

as in Definition 4.1. Apply the following finite rejection tests.

(A1) Reject if N violates identity, converse, or RCC5 composition on any triple of ports.

(A2) Reject if any assigned type is not a Hintikka type.

(A3) Reject if a local witness declaration for $\exists R.D$ points to a port not in relation R or not containing D.

(A4) Reject if a nonlocal request is not a PP or PPI request, or if it lacks a support direction.

(A5) Reject if an overlap map is not injective or fails to preserve type and network labels on the overlap.

(A6) Reject if a universal shadow omits an obligation induced by its source type.

(A7) Reject if side-context saturation has not been completed, i.e. if a side/tree or side/side pair labelled PP, PPI, or EQ is marked residual.

The retained labels form Σ_{C_0}.

K.5 Automaton construction

Construct the two-way alternating parity automaton $\mathcal{A}_{C_0}$ from Σ_{C_0} as follows.

(D1) The initial state guesses a root port p in the root bag with $C_0 \in \tau(p)$.

(D2) Local consistency states check all finite conditions already used in alphabet enumeration. These checks are redundant but useful for correctness: they ensure the input label belongs to Σ_{C_0}.

(D3) Overlap states move to parent and child bags and check overlap agreement.

(D4) For each existential formula in a port type, a witness state either verifies a same-bag witness or spawns a parity request state.

(D5) For each universal formula, a shadow state is spawned and propagated two-way through the decomposition.

(D6) Request states follow support branches until fulfilled.

(D7) The parity condition enforces that no request remains pending forever along an accepting branch.

Finally decide emptiness of $\mathcal{A}_{C_0}$ using the standard emptiness algorithm for finite two-way alternating parity tree automata. This is effective because the automaton is finite.

L Appendix L: expanded proof of Theorem A

Theorem A is the semantic normal-form theorem. This appendix expands the proof into individual local transformations.

L.1 Step 1: delete irrelevant elements

Apply witness thinning. The only subtle point is preservation of universals. Universals are preserved because deleting elements cannot add new counterexamples. Completeness of the closure under NNF complement supplies the converse direction: if a universal failed in the original model, its dual existential has a selected counter-witness and that counter-witness is retained. Thus the thinned model is truth-equivalent for all closure formulas.

L.2 Step 2: choose generated containment supports

For every generated element x and every true existential $\exists \mathsf{PP}.D$, choose one witness y with $x\mathsf{PP}y$ and $D \in \mathrm{tp}(y)$. This witness is a selected upward support. For every true existential $\exists \mathsf{PPI}.D$, choose one downward support. These supports define the generated vertical skeleton.

This skeleton need not be well-founded, rooted at a semantic top, or finite. It may contain infinite upward rays, downward rays, and regular repetitions. The automaton later handles the infinite behavior. The normal form only requires that supports are selected and tree-decomposed into bags.

L.3 Step 3: split multiple selected superparts

Suppose a generated occurrence x has two selected PP witnesses y and z that are not comparable. A single rooted containment tree cannot put both y and z on one parent chain above one node. The normal form therefore represents x by two local manifestations x_y and x_z in two bags. These manifestations are the same global occurrence because the bag overlap maps identify them. The two superparts y and z may then lie on two different local vertical branches.

This is not a weak-equality quotient. There are not two semantic elements x_y and x_z related by EQ. There is one occurrence whose local manifestations form a connected overlap class. This distinction is essential for avoiding the old blocking/equality error.

L.4 Step 4: insert side witnesses

For each selected DR or PO witness w of a source u, create a side context containing u, w and the relevant finite boundary. The boundary includes any vertical nodes needed to evaluate formulas in the closure at u or w: selected superparts/subparts, overlap manifestations, and live support endpoints. The context is then saturated.

L.5 Step 5: saturate and verticalize

Inspect every relation in the side context. If a side occurrence and a boundary occurrence are DR or PO, the pair may remain horizontal. If they are PP or PPI, insert a vertical stratum connecting them. If they are EQ, merge their manifestations by overlap identity. This operation may create new boundary pairs; repeat until no such pair remains.

If the forced comparison continues into an infinite ancestor or descendant tail, choose a bounded threshold representative of the repeating context type and turn the rest into a vertical request. This

is sound because the formulas only see closure types and RCC5 relations, and these are represented by the finite context class.

L.6 Step 6: prove residual frontier correctness

After saturation, suppose a pair is eligible to be forgotten as a residual frontier pair. By construction it is not EQ, PP, or PPI; otherwise saturation would have represented it explicitly. Since the RCC5 atoms are exactly five, the residual relation is DR or PO. This derives the residual frontier condition from saturation; it is not an assumption.

L.7 Step 7: bound the construction

The only recursive part is side-context insertion inside a side context. Such recursion is triggered by a proper subformula of the formula that created the current witness. Hence the modal-depth rank decreases. The number of ranks is bounded by $|\operatorname{Cl}(C_0)|$, and each rank has only finitely many labelled context types. This gives $K(C_0)$ and completes Theorem A.

M Appendix M: expanded proof of Theorem B

M.1 Extraction direction

Let Φ be a saturated split-forest presentation. The extraction algorithm produces a labelled tree T_Φ.

For each bag B in Φ:

- The port set is the finite set of local manifestations in the bag.
- The network N_B is the restriction of the original model's RCC5 map to those manifestations.
- The type map τ_B records the closure type of each manifestation.
- Overlap maps are copied from the tree decomposition.
- Witness declarations record the selected witnesses already present in the bag; if the selected witness is not in the bag and is vertical, a vertical request declaration is recorded.
- Shadows are recorded for all live or forgotten universal sources whose obligations must cross the bag boundary.

Every local consistency check is inherited from the model. Overlap agreement is inherited because overlap manifestations are the same occurrence. Saturation validity is inherited from the saturated presentation. Thus T_Φ is a valid patchwork abstraction.

M.2 Realization direction

Let T be a valid patchwork abstraction. The global domain is the set of overlap-equivalence classes of local ports. The main construction problem is to define a single total RCC5 relation on this domain. The proof does not choose relations route by route. Instead, it applies patchwork.

Take any finite set F of global occurrences. Choose a finite connected subtree of bags containing manifestations of all occurrences in F. The bag networks on this finite subtree agree on overlaps. By finite patchwork, their union has a satisfiable complete RCC5 completion. Thus every finite F has at least one relation labelling compatible with all bags on the chosen subtree. Compactness over the finite relation alphabet yields a total labelling for the countable domain.

M.3 Truth lemma: Boolean cases

For concept names, the interpretation is defined by types. If A is a concept name, put $x \in A^{\mathcal{I}_T}$ iff $A \in \mathrm{tp}(x)$. Negated concept names are handled by NNF complements. Conjunctions and disjunctions follow directly from type consistency.

M.4 Truth lemma: existential cases

If $\exists R.D \in \mathrm{tp}(x)$ and $R \in \{\mathsf{DR}, \mathsf{PO}\}$, validity supplies a same-bag or side-context witness y with relation R and $D \in \mathrm{tp}(y)$. If $R \in \{\mathsf{PP}, \mathsf{PPI}\}$ and no local witness exists, validity supplies a vertical support request. The parity condition ensures that the support branch reaches a target y whose exact recorded relation from x is R and whose type contains D. By induction, D holds at y. Thus the existential is true.

Conversely, if $\exists R.D \notin \mathrm{tp}(x)$, then $\forall R.\overline{D} \in \mathrm{tp}(x)$. The universal case below implies that every R-successor satisfies $\overline{D}$, so no R-successor satisfies D.

M.5 Truth lemma: universal cases

Let $\forall R.D \in \mathrm{tp}(x)$ and suppose $\rho_T(x, y) = R$. Pick a finite connected subtree of bags containing manifestations of x and y. Along this subtree, the abstraction either carries x as a live port or carries its shadow. The shadow contains the obligation (R, D) and the exact relation vector from x to every live port in the current bag. When y is introduced, the vector entry is R. Therefore validity forces $D \in \mathrm{tp}(y)$. By induction, $y \models D$.

Conversely, if $\forall R.D \notin \mathrm{tp}(x)$, then $\exists R.\overline{D} \in \mathrm{tp}(x)$. The existential case supplies an R-successor satisfying $\overline{D}$, so the universal is false. This completes the truth lemma.

N Appendix N: expanded automaton construction

The automaton can be written as an alternating automaton whose transition formula is a Boolean combination of local tests and moves. We describe the relevant states.

N.1 State families

q_{init}.	Selects a root port with C_0 in its type.
q_{loc}.	Checks that the current label is a legal bag label.
$q_{\mathrm{ov}}(p)$.	Checks that the occurrence represented by port p is continued or forgotten according to the overlap status bits, and that continuation is connected.
$q_{\exists}(p, \exists R.D)$.	Checks an existential formula at port p.
$q_{\forall}(p, \forall R.D)$.	Starts or updates a shadow obligation.
$q_{\mathrm{sh}}(s)$.	Carries shadow profile $s = (t, \vec{r}, \Omega)$ two-way.
$q_{\mathrm{req}}(p, R, D)$.	Follows a vertical support request.
q_{sat}.	Checks side-context saturation flags.

N.2 Transition clauses

The transition for q_{loc} is a finite conjunction of all local table checks. The transition for $q_{\exists}(p, \exists R.D)$ is:

$$\bigvee_{q \in P,\, N(p,q)=R,\, D \in \tau(q)} \top \quad \vee \quad \text{spawn } q_{\mathrm{req}}(p, R, D)$$

where the second disjunct is allowed only for $R \in \{\mathsf{PP}, \mathsf{PPI}\}$ and a declared support exists.

The transition for $q_{\mathrm{sh}}(s)$ universally branches to all adjacent bags in which the shadow must continue. In each adjacent bag it checks that the new vector agrees on overlaps and that every introduced port satisfying a guarded relation receives the required type formulas.

The transition for $q_{\mathrm{req}}(p, R, D)$ either accepts immediately if the current bag contains a witness port in relation R to the source and with D in its type, or moves along the declared support edge. If the source port is carried through an overlap, the state updates the port name. If the source is not live, the state carries the corresponding shadow.

N.3 Parity acceptance

Let E be the finite set of request kinds (R, D) with $R \in \{\mathsf{PP}, \mathsf{PPI}\}$ and $D \in \mathrm{Cl}(C_0)$. The generalized Buchi condition says that each request kind that remains pending infinitely often must be fulfilled infinitely often. A standard round-robin monitor over E gives a parity condition. Since E is finite, the parity index is finite and effectively computable.

O Appendix O: diagnostic examples

O.1 Infinite upward chain

The concept

$$\exists \mathsf{PP}.\top \sqcap \forall \mathsf{PP}.\exists \mathsf{PP}.\top$$

is represented by an infinite upward support branch. In the abstraction this is a repeating bag pattern, but the overlap classes in successive repetitions are distinct. The parity request for $\exists \mathsf{PP}.\top$ is fulfilled at the next upward support occurrence on every lap. No semantic equality cycle is introduced.

O.2 Two incomparable superparts

A concept requiring both an A-superpart and a B-superpart, while forbidding comparability between A and B witnesses, is represented by two bags sharing the source occurrence through overlap identity. One bag contains the A superpart, the other contains the B superpart. The source is one occurrence because the overlap classes are connected. The two superparts are related by whatever RCC5 relation is forced by the bag networks and patchwork; if the type constraints forbid all possibilities, the automaton rejects.

O.3 Side witness forced vertical

If a PO side witness becomes a forced proper part of a vertical boundary node by composition, saturation inserts a local vertical stratum. If the forced relation persists down an infinite descendant tail or up an infinite ancestor tail, threshold verticalization records the finite threshold and turns the remaining tail into a vertical request. This is exactly the case missed by unsaturated side bags.

O.4 The diagnostic $\mathsf{PPI};\mathsf{PP}$ exclusion

If a shared source has two superparts and one witness type forbids $\mathsf{EQ}, \mathsf{PP}, \mathsf{PPI}, \mathsf{PO}$ relations to a B witness, the only remaining relation would be DR. But $\mathsf{PPI};\mathsf{PP}$ excludes DR. A complete bag or finite patchwork union containing the source and both superparts therefore cannot be locally RCC5-consistent, and the automaton rejects.

P Appendix P: limitations and verification targets

The proof is modular enough that future machine or proof-assistant verification should focus on the following targets.

(V1) Formalize the RCC5 composition table and the patchwork theorem used here.

(V2) Formalize witness thinning for the closure language.

(V3) Formalize exact occurrence identity and prove strong equality.

(V4) Formalize the truth lemma for valid patchwork abstractions.

(V5) Implement the finite alphabet enumeration and local legality tests.

(V6) Implement the two-way automaton construction and parity request monitor.

The highest-risk mathematical input is the exact patchwork theorem for the chosen RCC5 semantics. If one replaces abstract composition-table semantics by a concrete topological semantics, this theorem must be replaced by an appropriate representation and amalgamation theorem.

Q Appendix Q: local legality tests in expanded form

This appendix lists the finite tests that define membership in the alphabet Σ_{C_0} in a way suitable for implementation. A candidate label is a tuple

$$\ell = (P, N, \tau, \mu, W, U, \Gamma, \mathsf{stat}, \mathsf{sat})$$

where stat stores introduction/continuation/forgetting status bits for ports and sat stores side-context saturation certificates.

Q.1 Network tests

The network tests are:

(N1) $P \subseteq \{1, \dots, K\}$.

(N2) $N : P^2 \to \mathsf{Rel}$ is total.

(N3) $N(p, p) = \mathsf{EQ}$ for all p.

(N4) If $p \neq q$, then $N(p, q) \neq \mathsf{EQ}$.

(N5) $N(q, p) = N(p, q)^{-1}$ for all p, q.

(N6) $N(p, r) \in \mathrm{Comp}(N(p, q), N(q, r))$ for all p, q, r.

These tests are exhaustive for local RCC5 consistency of a complete finite bag network under abstract composition-table semantics.

Q.2 Type tests

For every port p and type $\tau(p)$:

(T1) exactly one of $C, \overline{C}$ belongs to $\tau(p)$ for every $C \in \mathrm{Cl}(C_0)$;

(T2) Boolean clauses are respected;

(T3) every occurrence of an EQ modality has been normalized, or equivalently its normalized clause is present;

(T4) if $\forall R.D \in \tau(p)$ and $q \in P$ with $N(p,q) = R$, then $D \in \tau(q)$ unless the target q is explicitly delegated to a shadow check. In the implementation, same-bag universal targets can be checked immediately and shadow targets checked by U.

Q.3 Witness tests

For every $\exists R.D \in \tau(p)$ one of the following must be recorded.

(W1) A local witness $q \in P$ with $N(p,q) = R$ and $D \in \tau(q)$.

(W2) A side-context witness declaration, permitted only if the context is part of the saturated side data and contains a port q satisfying the same condition.

(W3) A vertical request declaration in Γ, permitted only for $R \in \{\mathsf{PP}, \mathsf{PPI}\}$.

No existential is allowed to be merely promised without being placed into one of these three categories.

Q.4 Overlap tests

For each parent or child overlap map $\mu : P \rightharpoonup P'$:

(O1) μ is injective.

(O2) For all p in the domain, $\tau'(\mu(p)) = \tau(p)$.

(O3) For all p, q in the domain, $N'(\mu(p), \mu(q)) = N(p,q)$.

(O4) Witness declarations that name only overlap ports agree after applying μ.

(O5) Shadow declarations whose source and live target ports are all in the overlap agree after applying μ.

These checks ensure that the overlap is an induced common subnetwork, as required by patchwork.

Q.5 Connectedness status tests

Each local port occurrence has one of the following statuses relative to each adjacent direction: absent, introduced, continued, or forgotten. A label is legal only if:

(C1) a port is introduced exactly once along every root-to-leaf branch;

(C2) once forgotten in a direction, it is not reintroduced below that direction;

(C3) if a port is continued to two children, those continuations are manifestations of the same global occurrence only when the current bag contains the branching overlap witnessing connectedness.

These are finite regular tree conditions, hence checkable by the two-way automaton. They are the bag-decomposition version of the running-intersection property.

Q.6 Saturation-certificate tests

The finite saturation certificate sat records, for every side witness and every boundary relation considered during saturation, one of the following outcomes:

(S1) retained horizontal DR or PO relation;

(S2) verticalized PP or PPI relation, with a local stratum or threshold support;

(S3) overlap identity;

(S4) recursive subcontext class of smaller rank.

A label is rejected if any considered side/tree or side/side pair has label PP, PPI, or EQ but is marked as residual. A label is also rejected if a recursive subcontext does not have strictly smaller rank. Thus infinite saturation loops cannot occur inside a finite bag label.

R Appendix R: proof of running-intersection and occurrence identity

The automaton must ensure that local port manifestations assemble into genuine global occurrences. This is the standard running-intersection condition from tree decompositions, but we spell it out because it replaces the old equality quotient.

Definition R.1 (Occurrence trace)**.** *For a local port occurrence (t, p) in an input tree, its trace is the set of all nodes s of the input tree such that some port of bag s is connected to (t, p) by a chain of overlap maps.*

Lemma R.2 (Trace connectedness)**.** *If the status tests (C1)–(C3) are accepted, then every occurrence trace is connected.*

Proof. Suppose a trace contained two nodes u, v but omitted a node w on the tree path between them. Along the path from u to v, the occurrence must be continued through every edge by overlap maps. If it is absent at w, then at one edge before w it was forgotten or not continued, and at a later edge it was reintroduced. This violates (C2). If it branches into two children without being present in the parent as a branching overlap, this violates (C3). Hence the trace is connected. □

Lemma R.3 (Identity is well-defined)**.** *The quotient of local port occurrences by overlap maps is an equivalence relation whose classes are precisely the connected traces accepted by the automaton.*

Proof. Overlap maps generate a reflexive, symmetric, transitive closure, hence an equivalence relation. By the previous lemma, each equivalence class appears on a connected set of bags. Conversely, any connected trace is generated by the overlap maps along its edges. Therefore the equivalence classes and traces coincide. □

Lemma R.4 (No accidental equality from blocking)**.** *If two port occurrences have identical labels but lie in different unconnected repetitions of a regular branch, they are distinct global occurrences.*

Proof. Global occurrence identity is generated only by overlap maps. Equal labels do not create an overlap map. If no connected overlap chain relates the two occurrences, they belong to distinct equivalence classes. The local network test forbids EQ between distinct ports in any bag, and patchwork preserves this strong equality. Hence profile equality and blocking repetition do not imply semantic equality. □

S Appendix S: exact use of two-wayness

This appendix explains why the proof chooses two-way automata rather than a one-way tree automaton.

A one-way automaton moving only from root to leaves must summarize everything about forgotten ancestors. That is possible in principle, but the proof would need a losslessness theorem for those summaries. Earlier drafts effectively tried to prove such a theorem with relation transformers and may-summaries, and the referee reports identified several gaps. The two-way automaton avoids this burden.

S.1 Checking an ancestor obligation

Suppose an occurrence x is introduced in an ancestor bag and later forgotten, while a descendant bag introduces y. To check $\forall R.D$ at x, the automaton can move from the descendant bag back along the decomposition path while carrying the shadow for x, or can spawn the shadow when x is forgotten and let it move forward and backward as needed. In either case, the automaton checks actual overlap agreement at each step. It is not forced to rely on a single irreversible summary.

S.2 Checking sibling interactions

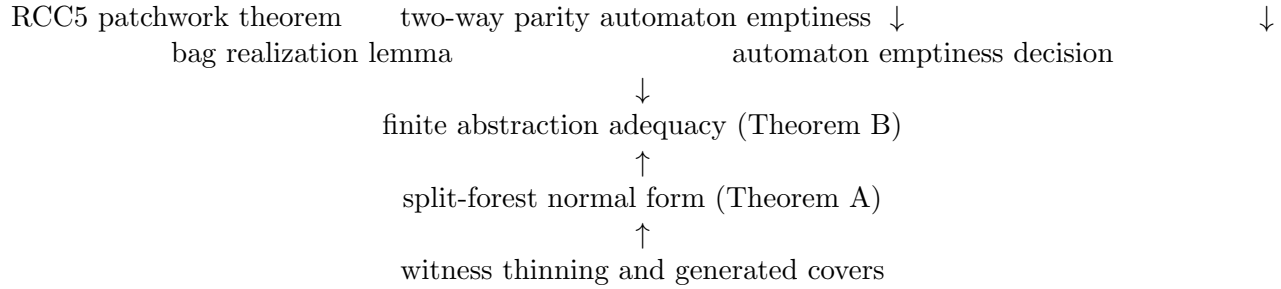
If a source in one child subtree has a target in another child subtree, the two-way automaton can move from one child to the parent interface and then to the other child, carrying the relevant shadow profile. The bag at the parent records the complete finite network on the shared frontier, and patchwork supplies the global completion. Thus sibling interactions are checked through actual finite interfaces, not through an implicit route computation.

S.3 No complexity claim

The use of two-way alternating parity automata may increase the worst-case complexity of the decision procedure. The theorem here is decidability, not an optimized upper bound. The gain is proof clarity: the automaton can revisit the finite information that the abstraction explicitly stores.

T Appendix T: dependency graph of the proof

The final decidability theorem has the following dependency graph.



The only semantic algebraic input is patchwork. The only automata-theoretic input is emptiness. All other steps are finite constructions over $\mathrm{Cl}(C_0)$, Rel, and the bounded bag width $K(C_0)$.

U Appendix U: what would invalidate the proof

For clarity, we also state what kinds of objections would be material.

(F1) If the exact RCC5 patchwork property in Theorem 2.5 failed for the intended abstract semantics, the proof would need a different global coherence theorem.

(F2) If side-context saturation did not have a computable finite quotient, the alphabet might not be finite.

(F3) If relation-vector shadows failed to preserve universal obligations across bag boundaries, the automaton could accept false abstractions.

(F4) If overlap identity did not enforce connected traces, blocking could again be confused with equality.

(F5) If vertical parity requests were not tied to concrete support branches, the automaton could accept unfulfilled eventualities.

The manuscript is organized so that each of these potential failure points is isolated in a named lemma or appendix.

V Conclusion

The final proof is modular. The split forest is the model normal form, finite patchwork bags are the abstraction of the normal form, and the two-way parity automaton is the decision procedure. This avoids both the semantic Hasse-diagram mistake and the later route-transformer coherence problem. The remaining non-elementary inputs are precisely identified: the RCC5 patchwork property and the standard emptiness theorem for two-way alternating parity tree automata.

References

[1] J. Renz and B. Nebel. Qualitative spatial reasoning using constraint calculi. Standard source for RCC composition tables and patchwork-style reasoning.

[2] M. Bodirsky and S. Wolfl. Homogeneous representations and patchwork properties for qualitative spatial and temporal calculi.

[3] M. Y. Vardi. Automata-theoretic techniques for modal and temporal logics, including two-way alternating automata and parity acceptance.